Complete Source Code Appendix

Complete repository source snapshot

[Student Name] | [Student ID]

[Department / Institution]

Network and System Programming

[Instructor Name]

13 November 2026

Common companion to both report editions

Source inventory and integrity

This companion contains the complete runtime source, build definition, tests, and demonstration/analysis scripts. Source comments are preserved in their original languages. Line numbers refer to original files; long lines wrap for readability. The ZIP stores unmodified bytes and a SHA-256 manifest. No generated binary, .git history, cache, or secret configuration is included.

Path	Original lines
.gitignore	6
Makefile	95
client/client.c	139
scripts/analyze_capture.py	91
scripts/demo.sh	326
server/server.c	259
server/system_info.c	107
server/system_info.h	7
shared/logging.c	38
shared/logging.h	17
shared/protocol.c	138
shared/protocol.h	22
tests/capture.sh	62
tests/common.py	201
tests/disconnect_probe.c	22
tests/fault_proc.c	18
tests/io_probe.c	52
tests/test_basic.py	75
tests/test_concurrency.py	71
tests/test_io.py	9
tests/test_lifecycle.py	39
tests/test_logging.py	26
tests/test_proc_errors.py	21
tests/test_robustness.py	52
tests/verify.py	27

.gitignore | part 1

Complete source listing (wrapped)

```
build/
bin/
lib/
__pycache__/
*.pyc

```

Makefile | part 1

Complete source listing (wrapped)

```
 1  CC := gcc
 2  DEBUG = 1
 3  VARIANT = protected
 4  ifeq ($(filter $(DEBUG),0 1),)
 5  $(error DEBUG must be 0 or 1)
 6  endif
 7  ifeq ($(filter $(VARIANT),protected baseline),)
 8  $(error VARIANT must be protected or baseline)
 9  endif
10  OUT := build/$(VARIANT)-debug$(DEBUG)
11  CPPFLAGS += -D_POSIX_C_SOURCE=200809L -DMONITOR_DEBUG=$(DEBUG) -Ishared -Iserver
12  CFLAGS ?= -O2 -g
13  CFLAGS += -std=c11 -Wall -Wextra -Wpedantic -Wformat=2 -Wshadow -Wconversion
14  ifeq ($(VARIANT),baseline)
15  CPPFLAGS += -DMONITOR_TEST_BASELINE
16  endif
17  LIB_OBJECTS := $(OUT)/shared/protocol.o $(OUT)/shared/logging.o
18  SERVER_OBJECTS := $(OUT)/server/server.o $(OUT)/server/system_info.o
19  CLIENT_OBJECTS := $(OUT)/client/client.o
20  ALL_OBJECTS := $(LIB_OBJECTS) $(SERVER_OBJECTS) $(CLIENT_OBJECTS) $(OUT)/tests/d
    isconnec
    t_probe.o $(OUT)/tests/fault_proc.o $(OUT)/tests/io_probe.o
21  LINK_LIB := -L$(OUT)/lib -lmonitor -Wl,-rpath,'$$ORIGIN/../lib'
22  .PHONY: all clean test test-concurrent test-robustness test-logging test-linkage
     test-li
    fecycle test-proc-errors test-io test-all baseline capture
23  all: $(OUT)/bin/server $(OUT)/bin/client
24
25  $(OUT)/lib/libmonitor.so: $(LIB_OBJECTS)
26    @mkdir -p $(@D)
27    $(CC) -shared $(LDFLAGS) -Wl,-soname,libmonitor.so -o $@ $^
28
29  $(OUT)/bin/server: $(SERVER_OBJECTS) $(OUT)/lib/libmonitor.so
30    @mkdir -p $(@D)
31    $(CC) $(LDFLAGS) -o $@ $(SERVER_OBJECTS) $(LINK_LIB)
32
33  $(OUT)/bin/client: $(CLIENT_OBJECTS) $(OUT)/lib/libmonitor.so
34    @mkdir -p $(@D)
35    $(CC) $(LDFLAGS) -o $@ $(CLIENT_OBJECTS) $(LINK_LIB)
36
37  $(OUT)/bin/disconnect_probe: $(OUT)/tests/disconnect_probe.o $(OUT)/lib/libmonit
    or.so
38    @mkdir -p $(@D)
39    $(CC) $(LDFLAGS) -o $@ $(OUT)/tests/disconnect_probe.o $(LINK_LIB)
40
41  $(OUT)/bin/server-fault: $(SERVER_OBJECTS) $(OUT)/tests/fault_proc.o $(OUT)/lib/
    libmonit
    or.so
```

Makefile | part 2

Complete source listing (wrapped)

```
42    @mkdir -p $(@D)
43    $(CC) $(LDFLAGS) -Wl,--wrap=fopen -o $@ $(SERVER_OBJECTS) $(OUT)/tests/fault_p
   roc.o $(
   LINK_LIB)
44
45  $(OUT)/bin/io_probe: $(OUT)/tests/io_probe.o $(OUT)/lib/libmonitor.so
46    @mkdir -p $(@D)
47    $(CC) $(LDFLAGS) -o $@ $(OUT)/tests/io_probe.o $(LINK_LIB)
48
49  $(OUT)/%.o: %.c
50    @mkdir -p $(@D)
51    $(CC) $(CPPFLAGS) $(CFLAGS) -fPIC -MMD -MP -c $< -o $@
52
53  baseline:
54    $(MAKE) VARIANT=baseline DEBUG=$(DEBUG) all build/baseline-debug$(DEBUG)/bin/d
   isconnec
   t_probe
55
56  test: all
57    python3 tests/test_basic.py --out $(OUT)
58
59  test-concurrent: all
60    python3 tests/test_concurrency.py --out $(OUT)
61
62  test-robustness: all $(OUT)/bin/disconnect_probe
63    $(MAKE) baseline
64    python3 tests/test_robustness.py --out $(OUT) --baseline build/baseline-debug$
   (DEBUG)
65
66  test-logging:
67    $(MAKE) DEBUG=1 all
68    $(MAKE) DEBUG=0 all
69    python3 tests/test_logging.py
70
71  test-linkage: all
72    ldd $(OUT)/bin/server
73    ldd $(OUT)/bin/client
74    readelf -d $(OUT)/bin/server
75    readelf -d $(OUT)/bin/client
76
77  test-lifecycle: all
78    python3 tests/test_lifecycle.py --out $(OUT)
79
80  test-proc-errors: $(OUT)/bin/server-fault
81    python3 tests/test_proc_errors.py --out $(OUT)
82
83  test-io: $(OUT)/bin/io_probe
```

Makefile | part 3

Complete source listing (wrapped)

```
84    python3 tests/test_io.py --out $(OUT)
85
86  test-all: test test-concurrent test-robustness test-logging test-linkage test-li
   fecycle
   test-proc-errors test-io
87
88  capture:
89    $(MAKE) DEBUG=1 all
90    bash tests/capture.sh
91
92  clean:
93    rm -rf build
94
95  -include $(ALL_OBJECTS:.o=.d)
```

client/client.c | part 1

Complete source listing (wrapped)

```
 1  #include "logging.h"
 2  #include "protocol.h"
 3  #include <errno.h>
 4  #include <getopt.h>
 5  #include <poll.h>
 6  #include <signal.h>
 7  #include <stdio.h>
 8  #include <stdlib.h>
 9  #include <string.h>
10  #include <time.h>
11  #include <unistd.h>
12
13  static void stop_signal(int number) { (void)number; mon_stop = 1; }
14
15  static void usage(const char *program)
16  {
17      fprintf(stderr,
18          "Usage: %s [--host numeric-IP] [--port 1..65535] [--log-level 0..2]\n"
19          "  [--command GET_STATUS|GET_CPU|GET_MEM|GET_UPTIME]\n"
20          "  [--monitor] [--interval-ms 100..600000] [--count N] [--timeout-ms 100
    ..600000
    ]\n"
21          "Without --command/--monitor, an interactive menu is shown.\n", program)
    ;
22  }
23
24  static int query(int fd, const char *command, int timeout_ms)
25  {
26      char request[MON_REQUEST_MAX + 1], response[MON_RESPONSE_MAX + 1];
27      int n = snprintf(request, sizeof(request), "%s\n", command);
28      if (n < 0 || n > MON_REQUEST_MAX) { LOG_ERROR("command too long"); return -1
    ; }
29      LOG_DEBUG("sending command=%s", command);
30      int result = mon_send_all(fd, request, (size_t)n, timeout_ms);
31      if (result == MON_LINE) result = mon_recv_line(fd, response, MON_RESPONSE_MA
    X, timeo
    ut_ms);
32      if (result != MON_LINE) {
33          if (!mon_stop) LOG_ERROR("query_failed result=%d errno=%d message=%s", r
    esult, e
    rrno, strerror(errno));
34          return -1;
35      }
36      puts(response);
37      fflush(stdout);
38      LOG_DEBUG("response_received");
39      if (strncmp(response, "OK ", 3) != 0) { LOG_ERROR("server response=%s", resp
    onse); r
    eturn -1; }
40      return 0;
```

client/client.c | part 2

Complete source listing (wrapped)

```
 41  }
 42
 43  static int monitor(int fd, int interval_ms, int count, int timeout_ms)
 44  {
 45      int completed = 0;
 46      while (!mon_stop && (count == 0 || completed < count)) {
 47          int64_t next = mon_now_ms() + interval_ms;
 48          if (query(fd, "GET_STATUS", timeout_ms) < 0) return mon_stop ? 0 : -1;
 49          ++completed;
 50          if (count && completed >= count) break;
 51          /* 使用 monotonic deadline；取樣耗時算入更新間隔。 */
 52          while (!mon_stop && mon_now_ms() < next) {
 53              int64_t left = next - mon_now_ms();
 54              if (left <= 0) break;
 55              poll(NULL, 0, (int)left);
 56          }
 57      }
 58      return 0;
 59  }
 60
 61  static int menu(int fd, int interval_ms, int count, int timeout_ms)
 62  {
 63      while (!mon_stop) {
 64          printf("\nLINUX SYSTEM MONITOR\n\n"
 65                 "1. Show System Overview\n2. Show CPU Usage\n3. Show Memory Usage
    \n"
 66                 "4. Show System Uptime\n5. Continuous Monitoring (Ctrl+C to exit)
    \n6. Exi
    t\n> ");
 67          fflush(stdout);
 68          /* 逐字元讀取避免 stdio 預讀與 poll 的 readiness 不一致。 */
 69          char input[32];
 70          size_t used = 0;
 71          int too_long = 0;
 72          for (;;) {
 73              int ready = mon_wait(STDIN_FILENO, POLLIN, -1);
 74              if (ready != MON_LINE) return mon_stop ? 0 : -1;
 75              char c;
 76              ssize_t n = read(STDIN_FILENO, &c, 1);
 77              if (n == 0) return 0;
 78              if (n < 0) { if (errno == EINTR) continue; return -1; }
 79              if (c == '\n') break;
 80              if (used < sizeof(input) - 1) input[used++] = c;
 81              else too_long = 1;
 82          }
 83          input[used] = '\0';
```

client/client.c | part 3

Complete source listing (wrapped)

```
 84          int choice;
 85          if (too_long || mon_parse_int(input, 1, 6, &choice) < 0) { puts("Please
    choose 1
     through 6."); continue; }
 86          if (choice == 6) return 0;
 87          if (choice == 5) return monitor(fd, interval_ms, count, timeout_ms);
 88          static const char *const commands[] = {"", "GET_STATUS", "GET_CPU", "GET
    _MEM", "
    GET_UPTIME"};
 89          if (query(fd, commands[choice], timeout_ms) < 0) return mon_stop ? 0 : -
    1;
 90      }
 91      return 0;
 92  }
 93
 94  int main(int argc, char **argv)
 95  {
 96      const char *host = "127.0.0.1", *port = "8080", *command = NULL;
 97      int level = 1, timeout_ms = 10000, interval_ms = 2000, count = 0, continuous
     = 0;
 98      static const struct option options[] = {
 99          {"host", required_argument, NULL, 'H'}, {"port", required_argument, NULL
    , 'p'},
100          {"log-level", required_argument, NULL, 'l'}, {"timeout-ms", required_arg
    ument, N
    ULL, 't'},
101          {"command", required_argument, NULL, 'c'}, {"monitor", no_argument, NULL
    , 'm'},
102          {"interval-ms", required_argument, NULL, 'i'}, {"count", required_argume
    nt, NULL
    , 'n'},
103          {"help", no_argument, NULL, 'h'}, {NULL, 0, NULL, 0}
104      };
105      int option, port_number;
106      while ((option = getopt_long(argc, argv, "H:p:l:t:c:mi:n:h", options, NULL))
     != -1)
    {
107          switch (option) {
108          case 'H': host = optarg; break;
109          case 'p': if (mon_parse_int(optarg, 1, 65535, &port_number)) goto invali
    d; port
    = optarg; break;
110          case 'l': if (mon_parse_int(optarg, 0, 2, &level)) goto invalid; break;
111          case 't': if (mon_parse_int(optarg, 100, 600000, &timeout_ms)) goto inva
    lid; bre
    ak;
112          case 'c': command = optarg; break;
113          case 'm': continuous = 1; break;
114          case 'i': if (mon_parse_int(optarg, 100, 600000, &interval_ms)) goto inv
    alid; br
    eak;
115          case 'n': if (mon_parse_int(optarg, 1, 1000000, &count)) goto invalid; b
    reak;
116          case 'h': usage(argv[0]); return 0;
117          default: goto invalid;
118          }
119      }
```

client/client.c | part 4

Complete source listing (wrapped)

```
120      if (optind != argc || (command && continuous)) goto invalid;
121      mon_log_init("client", level);
122      struct sigaction action;
123      memset(&action, 0, sizeof(action)); action.sa_handler = stop_signal; sigempt
    yset(&ac
    tion.sa_mask);
124      if (sigaction(SIGINT, &action, NULL) < 0 || sigaction(SIGTERM, &action, NULL
    ) < 0) {
125          LOG_ERROR("signal: %s", strerror(errno)); return 1;
126      }
127      int fd = mon_connect(host, port, timeout_ms);
128      if (fd < 0) { LOG_ERROR("connect: %s", strerror(errno)); return mon_stop ? 0
     : 1; }
129      LOG_INFO("connected host=%s port=%s", host, port);
130      if (!command && !continuous) printf("Connected to %s:%s\n", host, port);
131      int result = command ? query(fd, command, timeout_ms) :
132                   continuous ? monitor(fd, interval_ms, count, timeout_ms) :
133                   menu(fd, interval_ms, count, timeout_ms);
134      close(fd);
135      LOG_INFO("disconnected");
136      return mon_stop ? 0 : result == 0 ? 0 : 1;
137  invalid:
138      usage(argv[0]); return 2;
139  }
```

scripts/analyze_capture.py | part 1

Complete source listing (wrapped)

```
  1 """重現此次 Ethernet／IPv4 TCP pcap 的分析，不需要第三方套件。"""
  2 import collections
  3 import datetime
  4 import hashlib
  5 import json
  6 from pathlib import Path
  7 import struct
  8
  9 folder = Path(__file__).resolve().parents[1] / 'docs/evidence/packet-capture'
 10 path = folder / 'monitor.pcap'
 11 data = path.read_bytes()
 12 assert data[:4] == b'\xd4\xc3\xb2\xa1', 'expected little-endian microsecond clas
    sic pcap
    '
 13 assert len(data) >= 24
 14 _, major, minor, _, _, snaplen, linktype = struct.unpack('<IHHIIII', data[:24])
 15 assert (major, minor, linktype) == (2, 4, 1), 'this analysis expects Ethernet pc
    ap'
 16 offset, packets = 24, []
 17 while offset < len(data):
 18     assert offset + 16 <= len(data), 'truncated packet header'
 19     seconds, micros, saved, original = struct.unpack_from('<IIII', data, offset)
 20     offset += 16
 21     assert offset + saved <= len(data) and saved == original, 'truncated packet'
 22     frame = data[offset:offset + saved]
 23     offset += saved
 24     assert frame[12:14] == b'\x08\x00', 'expected IPv4 Ethernet frame'
 25     ip = frame[14:]
 26     assert ip[0] >> 4 == 4 and ip[9] == 6, 'expected IPv4 TCP'
 27     ip_size = (ip[0] & 15) * 4
 28     total = struct.unpack_from('!H', ip, 2)[0]
 29     tcp = ip[ip_size:total]
 30     source, destination, seq, ack = struct.unpack_from('!HHII', tcp)
 31     payload = tcp[(tcp[12] >> 4) * 4:]
 32     timestamp = datetime.datetime.fromtimestamp(seconds, datetime.timezone.utc)
 33     timestamp += datetime.timedelta(microseconds=micros)
 34     packets.append({'packet': len(packets) + 1, 'utc': timestamp.isoformat(times
    pec='mic
    roseconds'),
 35                     'time_us': seconds * 1000000 + micros,
 36                     'src': '.'.join(map(str, ip[12:16])), 'src_port': source,
 37                     'dst': '.'.join(map(str, ip[16:20])), 'dst_port': destinatio
    n,
 38                     'seq': seq, 'ack': ack, 'flags': tcp[13],
 39                     'payload_bytes': len(payload), 'payload': payload.decode('as
    cii'),
 40                     'payload_hex': payload.hex()})
 41
 42 flows = collections.defaultdict(list)
```

scripts/analyze_capture.py | part 2

Complete source listing (wrapped)

```
 for packet in packets:
     assert packet['src'] == packet['dst'] == '127.0.0.1'
     assert 8080 in (packet['src_port'], packet['dst_port'])
     client = packet['src_port'] if packet['dst_port'] == 8080 else packet['dst_p
ort']
     flows[client].append(packet)

 rows = []
 expected = {b'GET_STATUS\n': 'OK STATUS ', b'GET_CPU\n': 'OK CPU ',
             b'GET_MEM\n': 'OK MEM ', b'GET_UPTIME\n': 'OK UPTIME '}
 seen = set()
 for client, flow in flows.items():
     # 此次每筆訊息都在單一 segment；不將此假設套用至一般 TCP。
     assert [p['flags'] for p in flow[:3]] == [2, 18, 16], 'handshake missing'
     assert [p['flags'] for p in flow[-3:]] == [17, 17, 16], 'normal FIN closure
missing'
     assert all(not p['flags'] & 4 for p in flow), 'unexpected reset'
     syn, synack, ack = flow[:3]
     assert synack['ack'] == (syn['seq'] + 1) % 2**32
     assert ack['ack'] == (synack['seq'] + 1) % 2**32
     payloads = [p for p in flow if p['payload_bytes']]
     if not payloads:
         rows.append({'client_port': client, 'kind': 'readiness_probe', 'packets'
: len(fl
ow)})
         continue
     assert len(payloads) == 2
     request, response = payloads
     raw_request = request['payload'].encode('ascii')
     assert raw_request in expected and raw_request not in seen
     seen.add(raw_request)
     assert request['dst_port'] == 8080 and response['src_port'] == 8080
     assert response['payload'].startswith(expected[raw_request])
     assert response['payload'].endswith('\n')
     assert response['ack'] == (request['seq'] + request['payload_bytes']) % 2**3
2
     rows.append({'client_port': client, 'kind': 'query', 'command': raw_request.
decode()
.strip(),
                  'packets': len(flow), 'request_packet': request['packet'],
                  'response_packet': response['packet'], 'request_bytes': request
['payloa
d_bytes'],
                  'response_bytes': response['payload_bytes'], 'request_utc': req
uest['ut
c'],
                  'response_utc': response['utc'],
                  'request_to_response_ms': (response['time_us'] - request['time_
us']) /
1000,
                  'response': response['payload']})
 assert seen == set(expected), 'not all four queries captured'
```

scripts/analyze_capture.py | part 3

Complete source listing (wrapped)

```
 summary = {'pcap': path.name, 'bytes': len(data), 'sha256': hashlib.sha256(data)
.hexdige
st(),
            'link_type': 'EN10MB', 'snaplen': snaplen, 'packet_count': len(packet
s),
            'connections': len(flows), 'syn': sum(p['flags'] == 2 for p in packet
s),
            'syn_ack': sum(p['flags'] == 18 for p in packets),
            'fin': sum(bool(p['flags'] & 1) for p in packets),
            'rst': sum(bool(p['flags'] & 4) for p in packets),
            'payload_packets': sum(bool(p['payload_bytes']) for p in packets),
            'flows': rows, 'status': 'PASS'}
 (folder / 'analysis.json').write_text(json.dumps(summary, indent=2) + '\n')
 print(json.dumps(summary, indent=2))
```

scripts/demo.sh | part 1

Complete source listing (wrapped)

```
#!/usr/bin/env bash
# 分段執行真實展示；完整輸出保存在 evidence，終端機顯示適合截圖的摘要。
set -Eeuo pipefail
cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.."

usage() {
    cat <<'EOF'
Usage: bash scripts/demo.sh [--no-pause] [--live-capture] [STEP]

STEP: build, queries, concurrency, library, logging, robustness,
      capture, lifecycle, all (omit STEP for the interactive menu)

  --no-pause      Run a selected step or all steps without screenshot pauses.
  --live-capture  Guide a new capture in another terminal (capture step only).
  --help         Show this help.

DEMO_PORT=18080 changes the query/live-capture port (default: 8080).
The capture step normally analyzes the saved, dated project pcap.
The script never invokes sudo or takes desktop screenshots.
EOF
}

pause_enabled=1
live_capture=0
step=''
while (($#)); do
    case "$1" in
        --help|-h) usage; exit 0 ;;
        --no-pause) pause_enabled=0 ;;
        --live-capture) live_capture=1 ;;
        build|queries|concurrency|library|logging|robustness|capture|lifecycle|a
ll)
            if [[ -n "$step" ]]; then usage >&2; exit 2; fi
            step=$1 ;;
        *) usage >&2; exit 2 ;;
    esac
    shift
done
if [[ -z "$step" && ! -t 0 ]]; then
    echo 'Select a step explicitly when standard input is not a terminal.' >&2;
exit 2
fi
if ((live_capture)) && { [[ "$step" != capture ]] || ((pause_enabled == 0)) || [
[ ! -t 0
 ]]; }; then
    echo '--live-capture requires the capture step and an interactive terminal.'
 >&2; ex
it 2
```

scripts/demo.sh | part 2

Complete source listing (wrapped)

```
 43  fi
 44  if [[ ! -t 0 ]]; then pause_enabled=0; fi
 45  port=${DEMO_PORT:-8080}
 46  if [[ ! "$port" =~ ^[0-9]{1,5}$ ]]; then echo 'Invalid DEMO_PORT.' >&2; exit 2;
    fi
 47  port=$((10#$port))
 48  if ((port < 1024 || port > 65535)); then echo 'DEMO_PORT must be 1024..65535.' >
    &2; exit
     2; fi
 49
 50  session=docs/evidence/demo
 51  mkdir -p "$session"
 52  # 展示測試寫在 demo/tests，完整驗證保留在 verification。
 53  export MONITOR_EVIDENCE_ROOT="$session"
 54  server_pid=''
 55  step_log="$session/session.log"
 56  client=build/protected-debug1/bin/client
 57
 58  cleanup() {
 59      if [[ -n "$server_pid" ]]; then
 60          kill -TERM "$server_pid" 2>/dev/null || true
 61          wait "$server_pid" 2>/dev/null || true
 62          server_pid=''
 63      fi
 64  }
 65  fail() {
 66      local status=$1
 67      trap - ERR
 68      printf '\nFAILED (exit %s). Review %s\n' "$status" "$step_log" >&2
 69      exit "$status"
 70  }
 71  trap cleanup EXIT
 72  trap 'fail $?' ERR
 73  trap 'exit 130' INT
 74  trap 'exit 143' TERM
 75
 76  say() { printf '%s\n' "$*" | tee -a "$step_log"; }
 77  show_command() {
 78      { printf '$'; printf ' %q' "$@"; printf '\n'; } | tee -a "$step_log"
 79  }
 80  run() {
 81      show_command "$@"
 82      "$@" 2>&1 | tee -a "$step_log"
 83  }
 84  run_logged() {
 85      show_command "$@"
```

scripts/demo.sh | part 3

Complete source listing (wrapped)

```
     local status
     if "$@" >>"$step_log" 2>&1; then return 0; else status=$?; fi
     tail -n 45 "$step_log" >&2
     return "$status"
 }
 pause_for_screenshot() {
     if ((pause_enabled)); then
         printf '\nCapture this terminal window, then press Enter to continue...
'
         IFS= read -r _ || exit 0
     fi
 }
 ensure_build() { run_logged make DEBUG=1 VARIANT=protected all; }

 start_server() {
     local log="$session/$current_item-server.log"
     show_command build/protected-debug1/bin/server --port "$port" --log-level 2
--timeou
t-ms 30000
     build/protected-debug1/bin/server --port "$port" --log-level 2 --timeout-ms
30000 >"
$log" 2>&1 &
     server_pid=$!
     for _ in {1..50}; do
         if ! kill -0 "$server_pid" 2>/dev/null; then
             cat "$log" >&2
             say 'Server startup failed. Stop your old server or set DEMO_PORT.'
             return 1
         fi
         if grep -q 'listening address=' "$log"; then
             say "Server PID=$server_pid; port=$port; log=$log"
             return 0
         fi
         sleep .1
     done
     say 'Server startup timed out.'; return 1
 }
 stop_server() {
     show_command kill -TERM "$server_pid"
     kill -TERM "$server_pid"
     wait "$server_pid"
     say 'Server exited normally; its children have been reaped.'
     server_pid=''
 }
 query_traffic() {
     start_server
     for command in GET_STATUS GET_CPU GET_MEM GET_UPTIME; do
```

scripts/demo.sh | part 4

Complete source listing (wrapped)

```
        run "$client" --port "$port" --command "$command" --log-level 0
    done
    stop_server
}

test_summary() {
    python3 - "$step_log" "$1" <<'PY' | tee -a "$step_log"
import json
from pathlib import Path
import re
import sys
log, kind = Path(sys.argv[1]), sys.argv[2]
text = log.read_text()
matches = re.findall(r'^evidence: (.+)$', text, re.M)
assert matches, 'test evidence directory missing'
folder = Path(matches[-1])
result = json.loads((folder / 'result.json').read_text())
assert result['status'] == 'PASS', result
transcript = (folder / 'transcript.txt').read_text()
print(f'\nVerified result: {result["status"]}; evidence: {folder}')
if kind == 'concurrency':
    data = json.loads((folder / 'timings.json').read_text())
    print('Child PIDs:', ', '.join(map(str, data['pids'])))
    print('PID       Request start (ms)   Request end (ms)')
    for pid in data['pids']:
        print(f'{pid:<9} {data["server_starts_ms"][str(pid)]:<20} {data["server_
ends_ms"
][str(pid)]}')
    print(f'Concurrent: {data["concurrent_s"]:.6f} s; sequential: {data["sequent
ial_s"]:
.6f} s')
    print(f'Common overlap of all ten server intervals: {data["server_overlap_ms
"]} ms')
elif kind == 'robustness':
    for variant in ['baseline', 'protected']:
        segment = transcript.split(f'=== {variant}: Test A', 1)[1]
        if variant == 'baseline':
            segment = segment.split('=== protected: Test A', 1)[0]
        print(f'\n{variant.upper()}')
        for response in re.findall(r'^request=.* response=(.+)$', segment, re.M)
[:2]:
            print('Input validation:', response)
        observation = re.search(r'^observed_s=.+$', segment, re.M)
        assert observation
        print('Silent client:', observation[0])
        probe = segment.split('deterministic shared send path ===', 1)[1].split(
'TCP res
et integration', 1)[0]
        for line in probe.splitlines():
```

scripts/demo.sh | part 5

Complete source listing (wrapped)

```
            if line.startswith(('result=', 'exit=')):
                print('Closed-peer send:', line)
        reset = re.search(r'send_failed errno=\d+ message=[^\n]+', segment)
        if reset:
            print('TCP reset:', reset[0])
        print('Subsequent GET_MEM: verified by the test')
elif kind == 'logging':
    for line in transcript.splitlines():
        if line.startswith('=== compile'):
            print(line)
    print('Each server configuration checked client levels 0, 1, and 2.')
elif kind == 'lifecycle':
    for line in transcript.splitlines():
        if any(token in line for token in ['fd_before=', 'shutdown with held chi
ldren:',
 'restarted successfully']):
            print(line)
else:
    raise AssertionError(kind)
PY
}

capture_demo() {
    local pcap=docs/evidence/packet-capture/monitor.pcap
    if ((live_capture)); then
        pcap="$session/live-capture/monitor.pcap"
        mkdir -p "$session/live-capture"
        if [[ -e "$pcap" ]]; then
            say 'Move the previous live-capture/monitor.pcap before starting ano
ther cap
ture.'
            return 1
        fi
        say 'Terminal B: run this command and keep tcpdump running:'
        show_command sudo tcpdump -Z "$(id -un)" -i lo -nn -s 0 -U -w "$PWD/$pca
p" "tcp
port $port"
        printf '\nWhen Terminal B reports "listening on lo", press Enter here...
 '
        IFS= read -r _ || exit 0
        [[ -f "$pcap" ]] || { say 'No capture file found.'; return 1; }
        ensure_build
        query_traffic
        printf '\nStop tcpdump in Terminal B with Ctrl+C and photograph its stat
istics.\
nThen press Enter here... '
        IFS= read -r _ || exit 0
    else
        say 'Recorded capture from 2026-11-08; this step does not start a new ca
pture.'
```

scripts/demo.sh | part 6

Complete source listing (wrapped)

```
209      fi
210      run_logged env TZ=UTC tcpdump -nn -tttt -A -r "$pcap"
211      if ((live_capture == 0)); then
212          run_logged python3 scripts/analyze_capture.py
213          # 摘要直接來自 pcap 分析結果，不以固定展示數字代替實測。
214          python3 - <<'PY' | tee -a "$step_log"
215  import json
216  from pathlib import Path
217  folder = Path('docs/evidence/packet-capture')
218  data = json.loads((folder / 'analysis.json').read_text())
219  print('Saved pcap:', folder / data['pcap'])
220  print(f'Packets={data["packet_count"]}; connections={data["connections"]}; FIN={
    data["fi
    n"]}; RST={data["rst"]}')
221  for flow in data['flows']:
222      if flow['kind'] == 'query':
223          print(f'{flow["command"]:<12} client_port={flow["client_port"]} request=
    {flow["r
    equest_bytes"]} B '
224                f'response={flow["response_bytes"]} B delay={flow["request_to_resp
    onse_ms"
    ]:.3f} ms')
225  print('Operator-reported statistics from the original capture:')
226  print('\n'.join((folder / 'tcpdump-stats.txt').read_text().splitlines()[1:]))
227  PY
228          pause_for_screenshot
229      fi
230      if ((live_capture)); then
231          local decoded="$session/decoded-packets.txt"
232          show_command env TZ=UTC tcpdump -nn -tttt -A -r "$pcap"
233          env TZ=UTC tcpdump -nn -tttt -A -r "$pcap" >"$decoded" 2>>"$step_log"
234          cat "$decoded" >>"$step_log"
235          python3 - "$decoded" <<'PY' | tee -a "$step_log"
236  from pathlib import Path
237  import sys
238  text = Path(sys.argv[1]).read_text()
239  for name in ['STATUS', 'CPU', 'MEM', 'UPTIME']:
240      assert 'GET_' + name + '\n' in text, 'request missing: ' + name
241      assert 'OK ' + name + ' ' in text, 'response missing: ' + name
242  print('Verified all four request/response pairs in the new capture.')
243  PY
244          say 'New capture payloads (UTC):'
245          run env TZ=UTC tcpdump -nn -tttt -A -r "$pcap" 'tcp[13] & 8 != 0'
246      else
247          local memory_port
248          memory_port=$(python3 - <<'PY'
249  import json
```

scripts/demo.sh | part 7

Complete source listing (wrapped)

```
 from pathlib import Path
 data = json.loads(Path('docs/evidence/packet-capture/analysis.json').read_text()
)
 print(next(flow['client_port'] for flow in data['flows'] if flow.get('command')
== 'GET_
MEM'))
 PY
 )
         say 'Recorded GET_MEM connection: handshake, payloads, and FIN (UTC):'
         run env TZ=UTC tcpdump -nn -tttt -A -r "$pcap" "tcp port $memory_port"
     fi
 }

 run_step() {
     local item=$1
     current_item=$item
     step_log="$session/$item.log"
     : >"$step_log"
     say '============================================================'
     say "Linux System Monitor | ${item^^}"
     say "UTC: $(date -u +'%Y-%m-%d %H:%M:%S')"
     say "Full execution log: $step_log"
     say '============================================================'
     case "$item" in
         build)
             run make clean
             run make -j2 DEBUG=1 VARIANT=protected
             run ls -l build/protected-debug1/bin/client build/protected-debug1/b
in/serve
r build/protected-debug1/lib/libmonitor.so ;;
         queries) ensure_build; query_traffic ;;
         concurrency)
             run_logged make DEBUG=1 VARIANT=protected test-concurrent
             test_summary concurrency ;;
         library)
             ensure_build
             run ldd build/protected-debug1/bin/client
             run ldd build/protected-debug1/bin/server
             for binary in client server; do
                 run_logged readelf -d "build/protected-debug1/bin/$binary"
                 readelf -d "build/protected-debug1/bin/$binary" | grep -E 'NEEDE
D.*libmo
nitor|RUNPATH' | tee -a "$step_log"
             done ;;
         logging)
             run_logged make VARIANT=protected test-logging
             test_summary logging
             pause_for_screenshot
```

scripts/demo.sh | part 8

Complete source listing (wrapped)

```
291              start_server
292              for level in 0 1 2; do run "$client" --port "$port" --command GET_ME
    M --log-
    level "$level"; done
293              run build/protected-debug0/bin/client --port "$port" --command GET_M
    EM --log
    -level 2
294              stop_server ;;
295          robustness)
296              run_logged make DEBUG=1 VARIANT=protected test-robustness
297              test_summary robustness ;;
298          capture) capture_demo ;;
299          lifecycle)
300              run_logged make DEBUG=1 VARIANT=protected test-lifecycle
301              test_summary lifecycle ;;
302      esac
303      say "PASS: $item (all commands completed successfully)"
304      pause_for_screenshot
305  }
306
307  if [[ "$step" == all ]]; then
308      for item in build queries concurrency library logging robustness capture lif
    ecycle;
    do run_step "$item"; done
309  elif [[ -n "$step" ]]; then
310      run_step "$step"
311  else
312      while true; do
313          printf '\nLinux System Monitor - Demonstration\n\n'
314          printf '1. Clean Build\n2. System Information Queries\n3. Ten Concurrent
     Clients
    \n'
315          printf '4. Shared Library Verification\n5. Logging Controls\n6. Robustne
    ss Compa
    risons\n'
316          printf '7. Packet Capture (recorded)\n8. Process Lifecycle\n0. Exit\n\nS
    elect: '
317          IFS= read -r choice || break
318          case "$choice" in
319              1) run_step build ;; 2) run_step queries ;; 3) run_step concurrency
    ;;
320              4) run_step library ;; 5) run_step logging ;; 6) run_step robustness
     ;;
321              7) run_step capture ;; 8) run_step lifecycle ;; 0) break ;;
322              *) echo 'Please select 0 through 8.' ;;
323          esac
324      done
325  fi
326  printf '\nDemonstration records: %s\n' "$session"
```

server/server.c | part 1

Complete source listing (wrapped)

```
  1  #include "logging.h"
  2  #include "protocol.h"
  3  #include "system_info.h"
  4  #include <arpa/inet.h>
  5  #include <errno.h>
  6  #include <fcntl.h>
  7  #include <getopt.h>
  8  #include <netinet/in.h>
  9  #include <poll.h>
 10  #include <signal.h>
 11  #include <stdio.h>
 12  #include <stdlib.h>
 13  #include <string.h>
 14  #include <sys/socket.h>
 15  #include <sys/wait.h>
 16  #include <unistd.h>
 17
 18  #define MAX_CHILDREN 128
 19  static int signal_pipe[2] = {-1, -1};
 20  static pid_t children[MAX_CHILDREN];
 21  static volatile sig_atomic_t shutting_down;
 22
 23  static void parent_signal(int signal_number) /* parent signal handler */
 24  {
 25      int saved = errno;
 26      if (signal_number != SIGCHLD) shutting_down = 1;
 27      unsigned char byte = (unsigned char)signal_number;
 28      ssize_t ignored = write(signal_pipe[1], &byte, 1);
 29      (void)ignored;
 30      errno = saved;
 31  }
 32
 33  static void child_signal(int signal_number) /* child signal handler */
 34  {
 35      (void)signal_number;
 36      mon_stop = 1;
 37  }
 38
 39  static int install_signal(int number, void (*handler)(int)) /* send signal and i
    t's hand
    ler registered to Linux */
 40  {
 41      struct sigaction action;
 42      memset(&action, 0, sizeof(action));
 43      action.sa_handler = handler;
```

server/server.c | part 2

Complete source listing (wrapped)

```
44      sigemptyset(&action.sa_mask);
45      return sigaction(number, &action, NULL);
46  }
47
48  static int make_nonblocking(int fd) /* set file descriptor to non-blocking mode
    */
49  {
50      int flags = fcntl(fd, F_GETFL);
51      return flags < 0 ? -1 : fcntl(fd, F_SETFL, flags | O_NONBLOCK);
52  }
53
54  static void reap_children(void) /* reap all terminated child processes to preven
    t zombie
    s */
55  {
56      int status;
57      pid_t pid;
58      while ((pid = waitpid(-1, &status, WNOHANG)) > 0) {
59          for (size_t i = 0; i < MAX_CHILDREN; ++i) if (children[i] == pid) childr
    en[i] =
    0;
60          if (WIFSIGNALED(status)) LOG_ERROR("child_reaped child=%ld signal=%d", (
    long)pid
    , WTERMSIG(status));
61          else LOG_INFO("child_reaped child=%ld exit=%d", (long)pid, WEXITSTATUS(s
    tatus));
62      }
63  }
64
65  static int active_children(void) /* count the number of active child processes *
    /
66  {
67      int count = 0;
68      for (size_t i = 0; i < MAX_CHILDREN; ++i) if (children[i]) ++count;
69      return count;
70  }
71
72  static void shutdown_children(void) /* gracefully shutdown all child processes *
    /
73  {
74      for (size_t i = 0; i < MAX_CHILDREN; ++i) if (children[i]) kill(children[i],
     SIGTERM
    );
75      int64_t deadline = mon_now_ms() + 2000;
76      while (active_children() && mon_now_ms() < deadline) {
77          reap_children();
78          struct pollfd event = {.fd = signal_pipe[0], .events = POLLIN};
79          poll(&event, 1, 50);
80          unsigned char bytes[128];
81          while (read(signal_pipe[0], bytes, sizeof(bytes)) > 0) {}
82      }
83      for (size_t i = 0; i < MAX_CHILDREN; ++i) if (children[i]) kill(children[i],
     SIGKILL
```

server/server.c | part 3

Complete source listing (wrapped)

```
    );
 84      /* 關機階段確實等待所有子程序，避免遺留孤兒程序。 */
 85      for (size_t i = 0; i < MAX_CHILDREN; ++i) {
 86          if (!children[i]) continue;
 87          while (waitpid(children[i], NULL, 0) < 0 && errno == EINTR) {}
 88          children[i] = 0;
 89      }
 90  }
 91
 92  static int send_response(int fd, const char *response, int timeout_ms) /* send r
    esponse
    to client */
 93  {
 94      int result = mon_send_all(fd, response, strlen(response), timeout_ms);
 95      if (result != MON_LINE && result != MON_CANCELLED) {
 96          if (result == MON_TIMEOUT) LOG_ERROR("send_timeout");
 97          else LOG_ERROR("send_failed errno=%d message=%s", errno, strerror(errno)
    );
 98      }
 99      return result;
100  }
101
102  static void serve_client(int fd, int timeout_ms) /* handle client requests and s
    end resp
    onses */
103  {
104      LOG_INFO("connected");
105      int read_timeout = timeout_ms;
106  #ifdef MONITOR_TEST_BASELINE
107      read_timeout = -1;
108  #endif
109      while (!mon_stop) {
110          char request[MON_REQUEST_MAX + 1], response[MON_RESPONSE_MAX + 1];
111          int result = mon_recv_line(fd, request, MON_REQUEST_MAX, read_timeout);
112          if (result == MON_EOF || result == MON_CANCELLED) break;
113  #ifdef MONITOR_TEST_BASELINE
114          /* 超長資料只取已安全儲存的前綴，示範錯誤接受；回應後關閉。 */
115          int baseline_oversized = result == MON_TOO_LONG;
116          if (baseline_oversized) result = MON_LINE;
117  #endif
118          if (result != MON_LINE) {
119              const char *error = "ERR IO_ERROR\n";
120              if (result == MON_TIMEOUT) error = "ERR TIMEOUT\n";
121              if (result == MON_TOO_LONG) error = "ERR REQUEST_TOO_LONG\n";
122              if (result == MON_INVALID || result == MON_TRUNCATED) error = "ERR I
    NVALID_R
    EQUEST\n";
123              LOG_ERROR("request_rejected result=%d", result);
```

server/server.c | part 4

Complete source listing (wrapped)

```
124             send_response(fd, error, timeout_ms);
125             break;
126         }
127  #if MONITOR_DEBUG
128         int64_t start = mon_now_ms();
129  #endif
130         LOG_DEBUG("request_start mono_ms=%lld command=%s", (long long)start, req
    uest);
131         enum mon_command command = system_command(request);
132         if (command == CMD_INVALID) {
133             snprintf(response, sizeof(response), "ERR INVALID_REQUEST\n");
134             LOG_ERROR("invalid_command");
135         } else if (system_response(command, response, sizeof(response)) < 0) {
136             if (mon_stop) break;
137             snprintf(response, sizeof(response), "ERR SYSTEM_INFO\n");
138             LOG_ERROR("system_info_failed");
139         }
140         if (send_response(fd, response, timeout_ms) != MON_LINE) break;
141         LOG_DEBUG("request_end mono_ms=%lld elapsed_ms=%lld", (long long)mon_now
    _ms(),
142                   (long long)(mon_now_ms() - start));
143  #ifdef MONITOR_TEST_BASELINE
144         if (baseline_oversized) break;
145  #endif
146     }
147     close(fd);
148     LOG_INFO("disconnected");
149  }
150
151  static void usage(const char *program) /* print usage information for the progra
    m */
152  {
153     fprintf(stderr, "Usage: %s [--bind IPv4] [--port 1..65535] [--log-level 0..2
    ] [--tim
    eout-ms 100..600000]\n", program);
154  }
155
156  int main(int argc, char **argv)
157  {
158     const char *bind_address = "127.0.0.1";
159     int port = 8080, level = 1, timeout_ms = 10000;
160     static const struct option options[] = {
161         {"bind", required_argument, NULL, 'b'}, {"port", required_argument, NULL
    , 'p'},
162         {"log-level", required_argument, NULL, 'l'}, {"timeout-ms", required_arg
    ument, N
    ULL, 't'},
163         {"help", no_argument, NULL, 'h'}, {NULL, 0, NULL, 0}
164     };
165     int option;
```

server/server.c | part 5

Complete source listing (wrapped)

```
166      while ((option = getopt_long(argc, argv, "b:p:l:t:h", options, NULL)) != -1)
     {
167          switch (option) {
168          case 'b': bind_address = optarg; break;
169          case 'p': if (mon_parse_int(optarg, 1, 65535, &port)) goto invalid; brea
    k;
170          case 'l': if (mon_parse_int(optarg, 0, 2, &level)) goto invalid; break;
171          case 't': if (mon_parse_int(optarg, 100, 600000, &timeout_ms)) goto inva
    lid; bre
    ak;
172          case 'h': usage(argv[0]); return 0;
173          default: goto invalid;
174          }
175      }
176      if (optind != argc) goto invalid;
177      mon_log_init("parent", level);
178  #ifdef MONITOR_TEST_BASELINE
179      LOG_ERROR("TEST BASELINE: protections intentionally disabled; loopback only"
    );
180      if (strcmp(bind_address, "127.0.0.1") != 0) goto invalid;
181  #endif
182      struct sockaddr_in address = {.sin_family = AF_INET, .sin_port = htons((unsi
    gned sho
    rt)port)};
183      if (inet_pton(AF_INET, bind_address, &address.sin_addr) != 1) goto invalid;
184      int listener = socket(AF_INET, SOCK_STREAM, 0);
185      if (listener < 0) { LOG_ERROR("socket: %s", strerror(errno)); return 1; }
186      int reuse = 1;
187      if (setsockopt(listener, SOL_SOCKET, SO_REUSEADDR, &reuse, sizeof(reuse)) <
    0 ||
188          bind(listener, (struct sockaddr *)&address, sizeof(address)) < 0 ||
189          listen(listener, 64) < 0 || make_nonblocking(listener) < 0) {
190          LOG_ERROR("listen setup: %s", strerror(errno)); close(listener); return
    1;
191      }
192      if (pipe(signal_pipe) < 0) { LOG_ERROR("pipe: %s", strerror(errno)); close(l
    istener)
    ; return 1; }
193      if (make_nonblocking(signal_pipe[0]) < 0 || make_nonblocking(signal_pipe[1])
     < 0 ||
194          install_signal(SIGCHLD, parent_signal) < 0 || install_signal(SIGINT, par
    ent_sign
    al) < 0 ||
195          install_signal(SIGTERM, parent_signal) < 0 || install_signal(SIGPIPE, SI
    G_DFL) <
     0) {
196          LOG_ERROR("signal setup: %s", strerror(errno));
197          close(listener); close(signal_pipe[0]); close(signal_pipe[1]); return 1;
198      }
199      LOG_INFO("listening address=%s port=%d timeout_ms=%d", bind_address, port, t
    imeout_m
    s);
200      LOG_DEBUG("debug_enabled");
201      int exit_code = 0;
202      while (!shutting_down) {
203          reap_children();
```

server/server.c | part 6

Complete source listing (wrapped)

```
204          struct pollfd events[2] = {{.fd = listener, .events = POLLIN},
205                                     {.fd = signal_pipe[0], .events = POLLIN}};
206          int ready = poll(events, 2, -1);
207          if (ready < 0) {
208              if (errno == EINTR) continue;
209              LOG_ERROR("poll: %s", strerror(errno)); exit_code = 1; break;
210          }
211          if (events[1].revents) {
212              unsigned char bytes[128];
213              while (read(signal_pipe[0], bytes, sizeof(bytes)) > 0) {}
214              reap_children();
215          }
216          if (shutting_down || !(events[0].revents & POLLIN)) continue;
217          int connection = accept(listener, NULL, NULL);
218          if (connection < 0) {
219              if (errno != EINTR && errno != EAGAIN && errno != EWOULDBLOCK) {
220                  LOG_ERROR("accept: %s", strerror(errno));
221                  /* 資源不足時退出並清理，避免反覆 accept 形成 busy loop。 */
222                  exit_code = 1; break;
223              }
224              continue;
225          }
226          size_t slot = 0;
227          while (slot < MAX_CHILDREN && children[slot]) ++slot;
228          if (slot == MAX_CHILDREN) { LOG_ERROR("connection_limit"); close(connect
    ion); co
    ntinue; }
229          sigset_t blocked, previous;
230          sigemptyset(&blocked); sigaddset(&blocked, SIGINT); sigaddset(&blocked,
    SIGTERM)
    ; sigaddset(&blocked, SIGCHLD);
231          /* 處理信號；避免造成 child 回收和 PID 管理的 race condition */
232          if (sigprocmask(SIG_BLOCK, &blocked, &previous) < 0) {
233              LOG_ERROR("sigprocmask: %s", strerror(errno)); close(connection); ex
    it_code
    = 1; break;
234          }
235          pid_t child = fork();
236          /* Child 處裡連線 */
237          if (child == 0) {
238              close(listener); close(signal_pipe[0]); close(signal_pipe[1]);
239              if (install_signal(SIGCHLD, SIG_DFL) < 0 || install_signal(SIGINT, c
    hild_sig
    nal) < 0 ||
240                  install_signal(SIGTERM, child_signal) < 0 || sigprocmask(SIG_SET
    MASK, &p
    revious, NULL) < 0) {
241                  close(connection); _exit(1);
242              }
```

server/server.c | part 7

Complete source listing (wrapped)

```
243             mon_log_init("child", level);
244             serve_client(connection, timeout_ms);
245             _exit(0);
246         }
247         if (child < 0) LOG_ERROR("fork: %s", strerror(errno));
248         else { children[slot] = child; LOG_INFO("child_created child=%ld", (long
    )child);
     }
249         close(connection);
250         if (sigprocmask(SIG_SETMASK, &previous, NULL) < 0) { exit_code = 1; brea
    k; }
251     }
252     close(listener);
253     shutdown_children();
254     LOG_INFO("shutdown_complete active=%d", active_children());
255     close(signal_pipe[0]); close(signal_pipe[1]);
256     return exit_code;
257 invalid:
258     usage(argv[0]); return 2;
259 }
```

server/system_info.c | part 1

Complete source listing (wrapped)

```
 1  #include "system_info.h"
 2  #include "protocol.h"
 3  #include <errno.h>
 4  #include <stdio.h>
 5  #include <string.h>
 6  #include <time.h>
 7
 8  enum mon_command system_command(const char *request)
 9  {
10      static const char *const names[] = {"", "GET_STATUS", "GET_CPU", "GET_MEM",
   "GET_UPT
   IME"};
11      for (int i = CMD_STATUS; i <= CMD_UPTIME; ++i) {
12  #ifdef MONITOR_TEST_BASELINE
13          /* 對照版本只示範前綴誤判；不使用越界或記憶體破壞。 */
14          if (strncmp(request, names[i], strlen(names[i])) == 0) return (enum mon_
   command)
   i;
15  #else
16          if (strcmp(request, names[i]) == 0) return (enum mon_command)i;
17  #endif
18      }
19      return CMD_INVALID;
20  }
21
22  struct cpu_ticks { unsigned long long total, idle; };
23  static int read_cpu(struct cpu_ticks *ticks)
24  {
25      FILE *file = fopen("/proc/stat", "r");
26      if (!file) return -1;
27      unsigned long long user, nice, system, idle, iowait, irq, softirq, steal;
28      int n = fscanf(file, "cpu %llu %llu %llu %llu %llu %llu %llu %llu",
29                     &user, &nice, &system, &idle, &iowait, &irq, &softirq, &steal
   );
30      fclose(file);
31      if (n != 8) return -1;
32      /* guest 已包含在 user／nice，不重複加總；iowait 歸入閒置。 */
33      ticks->total = user + nice + system + idle + iowait + irq + softirq + steal;
34      ticks->idle = idle + iowait;
35      return 0;
36  }
37
38  static int cpu_usage(double *percent, long long *sample_ms)
39  {
40      struct cpu_ticks first, second;
41      if (read_cpu(&first) < 0) return -1;
42      int64_t start = mon_now_ms();
```

server/system_info.c | part 2

Complete source listing (wrapped)

```
43      struct timespec remaining = {.tv_sec = 0, .tv_nsec = 200000000};
44      while (nanosleep(&remaining, &remaining) < 0) {
45          if (errno != EINTR || mon_stop) return -1;
46      }
47      if (mon_stop || read_cpu(&second) < 0) return -1;
48      if (second.total <= first.total || second.idle < first.idle) return -1;
49      unsigned long long total = second.total - first.total;
50      unsigned long long idle = second.idle - first.idle;
51      if (idle > total) return -1;
52      *percent = 100.0 * (double)(total - idle) / (double)total;
53      *sample_ms = (long long)(mon_now_ms() - start);
54      return 0;
55  }
56
57  static int memory_info(unsigned long long *total, unsigned long long *available)
58  {
59      FILE *file = fopen("/proc/meminfo", "r");
60      if (!file) return -1;
61      char line[256];
62      int found_total = 0, found_available = 0;
63      while (fgets(line, sizeof(line), file)) {
64          if (sscanf(line, "MemTotal: %llu kB", total) == 1) found_total = 1;
65          if (sscanf(line, "MemAvailable: %llu kB", available) == 1) found_availab
   le = 1;
66      }
67      int failed = ferror(file);
68      fclose(file);
69      return !failed && found_total && found_available && *total && *available <=
   *total ?
    0 : -1;
70  }
71
72  static int uptime_info(double *seconds)
73  {
74      FILE *file = fopen("/proc/uptime", "r");
75      if (!file) return -1;
76      int n = fscanf(file, "%lf", seconds);
77      fclose(file);
78      return n == 1 && *seconds >= 0 ? 0 : -1;
79  }
80
81  int system_response(enum mon_command command, char *response, size_t capacity)
82  {
83      double cpu = 0, uptime = 0;
84      long long sample_ms = 0;
85      unsigned long long total = 0, available = 0;
```

server/system_info.c | part 3

Complete source listing (wrapped)

```
 86      if ((command == CMD_CPU || command == CMD_STATUS) && cpu_usage(&cpu, &sample
    _ms) < 0
    ) return -1;
 87      if ((command == CMD_MEM || command == CMD_STATUS) && memory_info(&total, &av
    ailable)
     < 0) return -1;
 88      if ((command == CMD_UPTIME || command == CMD_STATUS) && uptime_info(&uptime)
     < 0) re
    turn -1;
 89      unsigned long long used = total - available;
 90      double used_pct = total ? 100.0 * (double)used / (double)total : 0;
 91      int n;
 92      switch (command) {
 93      case CMD_CPU:
 94          n = snprintf(response, capacity, "OK CPU usage_pct=%.2f sample_ms=%lld\n
    ", cpu,
    sample_ms); break;
 95      case CMD_MEM:
 96          n = snprintf(response, capacity, "OK MEM total_kb=%llu available_kb=%llu
     used_kb
    =%llu used_pct=%.2f\n",
 97                       total, available, used, used_pct); break;
 98      case CMD_UPTIME:
 99          n = snprintf(response, capacity, "OK UPTIME seconds=%.2f\n", uptime); br
    eak;
100      case CMD_STATUS:
101          n = snprintf(response, capacity,
102                       "OK STATUS cpu_pct=%.2f sample_ms=%lld total_kb=%llu availa
    ble_kb=%
    llu used_kb=%llu used_pct=%.2f uptime_seconds=%.2f\n",
103                       cpu, sample_ms, total, available, used, used_pct, uptime);
    break;
104      default: return -1;
105      }
106      return n < 0 || (size_t)n >= capacity ? -1 : 0;
107  }
```

server/system_info.h | part 1

Complete source listing (wrapped)

```
#ifndef MONITOR_SYSTEM_INFO_H
#define MONITOR_SYSTEM_INFO_H
#include <stddef.h>
enum mon_command { CMD_INVALID, CMD_STATUS, CMD_CPU, CMD_MEM, CMD_UPTIME };
enum mon_command system_command(const char *request);
int system_response(enum mon_command command, char *response, size_t capacity);
#endif
```

shared/logging.c | part 1

Complete source listing (wrapped)

```
#include "logging.h"
#include <stdarg.h>
#include <stdio.h>
#include <time.h>
#include <unistd.h>

static const char *log_role = "monitor";
static int log_level;

void mon_log_init(const char *role, int level)
{
    log_role = role;
    log_level = level;
}

void mon_log(int level, const char *format, ...)
{
    if (level > log_level) return;
    static const char *const names[] = {"ERROR", "INFO", "DEBUG"};
    char line[1024];
    struct timespec now;
    clock_gettime(CLOCK_REALTIME, &now);
    int prefix = snprintf(line, sizeof(line), "%lld.%03ld pid=%ld role=%s %s ",
                          (long long)now.tv_sec, now.tv_nsec / 1000000,
                          (long)getpid(), log_role, names[level]);
    if (prefix < 0 || (size_t)prefix >= sizeof(line) - 2) return;
    va_list args;
    va_start(args, format);
    int n = vsnprintf(line + prefix, sizeof(line) - (size_t)prefix - 1, format,
args);
    va_end(args);
    if (n < 0) return;
    size_t used = (size_t)prefix + (size_t)n;
    if (used > sizeof(line) - 2) used = sizeof(line) - 2;
    line[used++] = '\n';
    /* 單次寫出，降低多個子程序的日誌交錯機率。 */
    ssize_t ignored = write(STDERR_FILENO, line, used);
    (void)ignored;
}
```

shared/logging.h | part 1

Complete source listing (wrapped)

```
 1  #ifndef MONITOR_LOGGING_H
 2  #define MONITOR_LOGGING_H
 3
 4  enum { MON_ERROR = 0, MON_INFO = 1, MON_DEBUG = 2 };
 5  void mon_log_init(const char *role, int level);
 6  void mon_log(int level, const char *format, ...) __attribute__((format(printf, 2
   , 3)));
 7  #ifndef MONITOR_DEBUG
 8  #define MONITOR_DEBUG 0
 9  #endif
10  #if MONITOR_DEBUG
11  #define LOG_DEBUG(...) mon_log(MON_DEBUG, __VA_ARGS__)
12  #else
13  #define LOG_DEBUG(...) ((void)0)
14  #endif
15  #define LOG_INFO(...) mon_log(MON_INFO, __VA_ARGS__)
16  #define LOG_ERROR(...) mon_log(MON_ERROR, __VA_ARGS__)
17  #endif
```

shared/protocol.c | part 1

Complete source listing (wrapped)

```
  1  #include "protocol.h"
  2  #include <errno.h>
  3  #include <fcntl.h>
  4  #include <limits.h>
  5  #include <netdb.h>
  6  #include <poll.h>
  7  #include <stdlib.h>
  8  #include <string.h>
  9  #include <sys/socket.h>
 10  #include <time.h>
 11  #include <unistd.h>
 12
 13  volatile sig_atomic_t mon_stop;
 14
 15  int64_t mon_now_ms(void)
 16  {
 17      struct timespec ts;
 18      if (clock_gettime(CLOCK_MONOTONIC, &ts) < 0) return 0;
 19      return (int64_t)ts.tv_sec * 1000 + ts.tv_nsec / 1000000;
 20  }
 21
 22  int mon_wait(int fd, short events, int64_t deadline)
 23  {
 24      for (;;) {
 25          if (mon_stop) return MON_CANCELLED;
 26          /* 限制單次等待，避免訊號剛好在檢查旗標後到達造成漏喚醒。 */
 27          int wait_ms = 200;
 28          if (deadline >= 0) {
 29              int64_t left = deadline - mon_now_ms();
 30              if (left <= 0) return MON_TIMEOUT;
 31              wait_ms = left > 200 ? 200 : (int)left;
 32          }
 33          struct pollfd p = {.fd = fd, .events = events};
 34          int result = poll(&p, 1, wait_ms);
 35          if (result > 0) {
 36              if (p.revents & POLLNVAL) { errno = EBADF; return MON_IO_ERROR; }
 37              return MON_LINE; /* HUP／ERR 交由 recv／send 取得具體結果。 */
 38          }
 39          if (result == 0) continue;
 40          if (errno != EINTR) return MON_IO_ERROR;
 41      }
 42  }
 43
 44  int mon_recv_line(int fd, char *buffer, size_t wire_limit, int timeout_ms)
```

shared/protocol.c | part 2

Complete source listing (wrapped)

```
 {
     int64_t deadline = timeout_ms < 0 ? -1 : mon_now_ms() + timeout_ms;
     size_t used = 0;
     buffer[0] = '\0';
     for (;;) {
         int ready = mon_wait(fd, POLLIN, deadline);
         if (ready != MON_LINE) return ready;
         unsigned char c;
         ssize_t n = recv(fd, &c, 1, MSG_DONTWAIT);
         /*
         n > 0: success, return byte,
         n = 0: EOF, return MON_EOF,
         n < 0: error, return MON_IO_ERROR,
         */
         if (n == 0) return used ? MON_TRUNCATED : MON_EOF;
         if (n < 0) {
             if (errno == EINTR || errno == EAGAIN || errno == EWOULDBLOCK) conti
nue;
             return MON_IO_ERROR;
         }
         if (c == '\n') { buffer[used] = '\0'; return MON_LINE; }
         if (c == '\0' || c == '\r' || c < 32 || c > 126) return MON_INVALID;
         if (used >= wire_limit - 1) return MON_TOO_LONG;
         buffer[used++] = (char)c;
         buffer[used] = '\0';
     }
 }

 int mon_send_all(int fd, const char *buffer, size_t length, int timeout_ms)
 {
     int64_t deadline = timeout_ms < 0 ? -1 : mon_now_ms() + timeout_ms;
     size_t sent = 0;
     while (sent < length) {
         int ready = mon_wait(fd, POLLOUT, deadline);
         if (ready != MON_LINE) return ready;
         int flags = MSG_DONTWAIT;
 #ifndef MONITOR_TEST_BASELINE
         flags |= MSG_NOSIGNAL;
 #endif
         ssize_t n = send(fd, buffer + sent, length - sent, flags);
         if (n < 0) {
             if (errno == EINTR || errno == EAGAIN || errno == EWOULDBLOCK) conti
nue;
             return MON_IO_ERROR;
         }
         if (n == 0) { errno = EPIPE; return MON_IO_ERROR; }
```

shared/protocol.c | part 3

Complete source listing (wrapped)

```
 89          sent += (size_t)n;
 90      }
 91      return MON_LINE;
 92  }
 93
 94  int mon_parse_int(const char *text, int minimum, int maximum, int *value)
 95  {
 96      if (!text || !*text) return -1;
 97      for (const char *p = text; *p; ++p) if (*p < '0' || *p > '9') return -1;
 98      char *end;
 99      errno = 0;
100      long n = strtol(text, &end, 10);
101      if (errno || *end || n < minimum || n > maximum) return -1;
102      *value = (int)n;
103      return 0;
104  }
105
106  int mon_connect(const char *host, const char *port, int timeout_ms)
107  {
108      /* 僅接受數字位址，避免同步 DNS 解析突破連線期限。 */
109      struct addrinfo hints = {.ai_family = AF_UNSPEC, .ai_socktype = SOCK_STREAM,
110                               .ai_flags = AI_NUMERICHOST | AI_NUMERICSERV};
111      struct addrinfo *addresses;
112      if (getaddrinfo(host, port, &hints, &addresses) != 0) { errno = EINVAL; retu
    rn -1; }
113      int fd = -1, saved = ECONNREFUSED;
114      int64_t deadline = mon_now_ms() + timeout_ms;
115      for (struct addrinfo *a = addresses; a && !mon_stop; a = a->ai_next) {
116          fd = socket(a->ai_family, a->ai_socktype, a->ai_protocol);
117          if (fd < 0) { saved = errno; continue; }
118          int flags = fcntl(fd, F_GETFL);
119          if (flags < 0 || fcntl(fd, F_SETFL, flags | O_NONBLOCK) < 0) {
120              saved = errno; close(fd); fd = -1; continue;
121          }
122          int result = connect(fd, a->ai_addr, a->ai_addrlen);
123          if (result < 0 && errno == EINPROGRESS) {
124              int ready = mon_wait(fd, POLLOUT, deadline);
125              if (ready == MON_LINE) {
126                  int error = 0;
127                  socklen_t size = sizeof(error);
128                  result = getsockopt(fd, SOL_SOCKET, SO_ERROR, &error, &size);
129                  if (result == 0 && error) { errno = error; result = -1; }
130              } else { errno = ready == MON_TIMEOUT ? ETIMEDOUT : EINTR; result =
    -1; }
131          }
132          if (result == 0) break;
```

shared/protocol.c | part 4

Complete source listing (wrapped)

```
133          saved = errno; close(fd); fd = -1;
134      }
135      freeaddrinfo(addresses);
136      if (fd < 0) errno = saved;
137      return fd;
138  }
```

shared/protocol.h | part 1

Complete source listing (wrapped)

```
#ifndef MONITOR_PROTOCOL_H
#define MONITOR_PROTOCOL_H
#include <signal.h>
#include <stddef.h>
#include <stdint.h>

/* 長度包含換行；額外一個 byte 保留給 C 字串結尾。 */
#define MON_REQUEST_MAX 128
#define MON_RESPONSE_MAX 2048
enum mon_result {
    MON_LINE = 1, MON_EOF = 0, MON_IO_ERROR = -1, MON_TIMEOUT = -2,
    MON_TOO_LONG = -3, MON_INVALID = -4, MON_TRUNCATED = -5,
    MON_CANCELLED = -6
};
extern volatile sig_atomic_t mon_stop;
int64_t mon_now_ms(void);
int mon_wait(int fd, short events, int64_t deadline);
int mon_recv_line(int fd, char *buffer, size_t wire_limit, int timeout_ms);
int mon_send_all(int fd, const char *buffer, size_t length, int timeout_ms);
int mon_parse_int(const char *text, int minimum, int maximum, int *value);
int mon_connect(const char *host, const char *port, int timeout_ms);
#endif
```

tests/capture.sh | part 1

Complete source listing (wrapped)

```
  1  #!/usr/bin/env bash
  2  # 預設不呼叫 sudo；僅在使用者明確設定 MONITOR_CAPTURE_SUDO=1 時使用。
  3  set -euo pipefail
  4  cd -- "$(dirname -- "${BASH_SOURCE[0]}")/.."
  5  port=${MONITOR_CAPTURE_PORT:-8080}
  6  case "$port" in ''|*[!0-9]*) echo 'Invalid capture port' >&2; exit 2;; esac
  7  if ((port < 1024 || port > 65535)); then echo 'Capture port must be 1024..65535'
     >&2; ex
    it 2; fi
  8  evidence=docs/evidence/demo/live-capture
  9  if [[ -e "$evidence/monitor.pcap" ]]; then
 10      echo 'Move the previous demo/live-capture/monitor.pcap before starting anoth
    er captu
    re.' >&2; exit 2
 11  fi
 12  mkdir -p "$evidence"
 13  exec > >(tee "$evidence/transcript.txt") 2>&1
 14  set -x
 15  server_pid=''
 16  capture_pid=''
 17  cleanup() {
 18      set +e
 19      if [[ -n "$server_pid" ]]; then kill -TERM "$server_pid" 2>/dev/null; wait "
    $server_
    pid"; fi
 20      if [[ -n "$capture_pid" ]]; then kill -INT "$capture_pid" 2>/dev/null; wait
    "$captur
    e_pid"; fi
 21  }
 22  trap cleanup EXIT
 23  trap 'exit 130' INT
 24  trap 'exit 143' TERM
 25  capture_command=(tcpdump)
 26  if [[ ${MONITOR_CAPTURE_SUDO:-0} == 1 ]]; then capture_command=(sudo -n tcpdump
    -Z "$(id
     -un)"); fi
 27  "${capture_command[@]}" -i lo -nn -s 0 -U -w "$evidence/monitor.pcap" "tcp port
    $port" 2
    >"$evidence/tcpdump.log" &
 28  capture_pid=$!
 29  for _ in {1..50}; do
 30      if ! kill -0 "$capture_pid" 2>/dev/null; then
 31          cat "$evidence/tcpdump.log"
 32          echo 'BLOCKED: tcpdump could not start; capture permission is required.'
 33          exit 77
 34      fi
 35      if grep -q 'listening on' "$evidence/tcpdump.log"; then break; fi
 36      sleep 0.1
 37  done
 38  if ! grep -q 'listening on' "$evidence/tcpdump.log"; then echo 'FAIL: capture st
    artup ti
```

tests/capture.sh | part 2

Complete source listing (wrapped)

```
   meout'; exit 1; fi
39  build/protected-debug1/bin/server --port "$port" --log-level 2 2>"$evidence/serv
   er.log"
   &
40  server_pid=$!
41  for _ in {1..50}; do
42      if ! kill -0 "$server_pid" 2>/dev/null; then cat "$evidence/server.log"; exi
   t 1; fi
43      if grep -q 'listening address=' "$evidence/server.log"; then break; fi
44      sleep 0.1
45  done
46  for command in GET_STATUS GET_CPU GET_MEM GET_UPTIME; do
47      build/protected-debug1/bin/client --port "$port" --command "$command" --log-
   level 2
48  done
49  kill -TERM "$server_pid"
50  wait "$server_pid"
51  server_pid=''
52  # 等待 loopback 的最後 FIN 進入擷取程序。
53  sleep 1
54  kill -INT "$capture_pid"
55  wait "$capture_pid"
56  capture_pid=''
57  tcpdump -nn -tttt -A -r "$evidence/monitor.pcap" > "$evidence/packets.txt" 2> "$
   evidence
   /read.log"
58  cat "$evidence/tcpdump.log"
59  if ! grep -q 'GET_MEM' "$evidence/packets.txt" || ! grep -q 'OK MEM' "$evidence/
   packets.
   txt"; then
60      echo 'FAIL: request/response missing in capture'; exit 1
61  fi
62  echo "PASS: capture; evidence=$evidence"
```

tests/common.py | part 1

Complete source listing (wrapped)

```
  1  """Linux 整合測試共用工具；所有結果來自本次執行。"""
  2  import argparse
  3  import contextlib
  4  import datetime
  5  import json
  6  import os
  7  from pathlib import Path
  8  import shlex
  9  import signal
 10  import shutil
 11  import socket
 12  import subprocess
 13  import sys
 14  import time
 15  import traceback
 16
 17  ROOT = Path(__file__).resolve().parents[1]
 18
 19
 20  class Tee:
 21      def __init__(self, *files):
 22          self.files = files
 23
 24      def write(self, text):
 25          for file in self.files:
 26              file.write(text)
 27              file.flush()
 28
 29      def flush(self):
 30          for file in self.files:
 31              file.flush()
 32
 33
 34  def command(args, *, check=True, timeout=20, **kwargs):
 35      args = [str(arg) for arg in args]
 36      print('$', shlex.join(args), flush=True)
 37      result = subprocess.run(args, text=True, capture_output=True, timeout=timeou
    t, **kwa
    rgs)
 38      if result.stdout:
 39          print(result.stdout, end='' if result.stdout.endswith('\n') else '\n')
 40      if result.stderr:
 41          print(result.stderr, end='' if result.stderr.endswith('\n') else '\n')
 42      print(f'exit={result.returncode}')
 43      if check and result.returncode:
```

tests/common.py | part 2

Complete source listing (wrapped)

```
         raise AssertionError(f'command failed: {args}, exit={result.returncode}'
)
     return result


 def wait_for(predicate, timeout=4):
     deadline = time.monotonic() + timeout
     while time.monotonic() < deadline:
         value = predicate()
         if value:
             return value
         time.sleep(.02)
     raise AssertionError('condition not reached before deadline')


 def child_pids(pid):
     try:
         return [int(x) for x in Path(f'/proc/{pid}/task/{pid}/children').read_te
xt().spl
it()]
     except FileNotFoundError:
         return []


 def receive(sock):
     data = bytearray()
     while not data.endswith(b'\n'):
         part = sock.recv(1)
         if not part:
             break
         data.extend(part)
         if len(data) > 4096:
             raise AssertionError('response exceeded test limit')
     return data.decode('ascii')


 def request(sock, payload):
     sock.sendall(payload)
     result = receive(sock)
     print(f'request={payload[:150]!r} bytes={len(payload)} response={result.rstr
ip()!r}'
)
     return result


 def fields(response):
     return dict(item.split('=', 1) for item in response.split()[2:])
```

tests/common.py | part 3

Complete source listing (wrapped)

```
 86
 87
 88  class Server:
 89      def __init__(self, out, evidence, *, level=2, timeout_ms=3000, port=None,
 90                   binary='server', env=None, label='server'):
 91          self.out = ROOT / out
 92          self.evidence = evidence
 93          self.level = level
 94          self.timeout_ms = timeout_ms
 95          self.binary = binary
 96          self.env = env
 97          self.label = label
 98          if port is None:
 99              with socket.socket() as temporary:
100                  temporary.bind(('127.0.0.1', 0))
101                  port = temporary.getsockname()[1]
102          self.port = port
103          self.process = None
104
105      def __enter__(self):
106          self.log_path = self.evidence / f'{self.label}.log'
107          # 同一測試若啟動多個未命名的 Server，使用短序號避免覆寫。
108          sequence = 2
109          while self.log_path.exists():
110              self.log_path = self.evidence / f'{self.label}-{sequence:02d}.log'
111              sequence += 1
112          self.log_file = self.log_path.open('w')
113          args = [str(self.out / 'bin' / self.binary), '--port', str(self.port),
114                  '--log-level', str(self.level), '--timeout-ms', str(self.timeout
    _ms)]
115          print('$', shlex.join(args), '2>', self.log_path.relative_to(ROOT))
116          self.process = subprocess.Popen(args, stdout=self.log_file, stderr=self.
    log_file
    , env=self.env)
117          try:
118              deadline = time.monotonic() + 4
119              while True:
120                  if self.process.poll() is not None:
121                      raise AssertionError(f'server exited: {self.log_path.read_te
    xt()}')
122                  try:
123                      with self.connect():
124                          break
125                  except ConnectionRefusedError:
126                      if time.monotonic() >= deadline:
127                          raise AssertionError('server startup timed out')
128                      time.sleep(.02)
```

tests/common.py | part 4

Complete source listing (wrapped)

```
            # 等待 readiness probe 建立的子程序完成回收。
            time.sleep(.05)
            wait_for(lambda: not child_pids(self.process.pid))
            print(f'server_parent={self.process.pid} port={self.port}')
            return self
        except BaseException:
            self.__exit__(None, None, None)
            raise

    def connect(self):
        return socket.create_connection(('127.0.0.1', self.port), timeout=3)

    def client(self, *args, **kwargs):
        return command([self.out / 'bin/client', '--port', self.port, *args], **
    kwargs)

    def logs(self):
        return self.log_path.read_text()

    def __exit__(self, *exc):
        if self.process is not None:
            if self.process.poll() is None:
                self.process.terminate()
            try:
                self.process.wait(timeout=4)
            except subprocess.TimeoutExpired:
                self.process.kill()
                self.process.wait()
                raise AssertionError('server shutdown exceeded deadline')
            finally:
                self.log_file.close()
            print(f'server_exit={self.process.returncode} log={self.log_path.rel
    ative_to
    (ROOT)}')
            if self.process.returncode != 0 and exc[0] is None:
                raise AssertionError('server did not exit cleanly')


 def run_test(name, function, extra_arguments=None):
    parser = argparse.ArgumentParser()
    parser.add_argument('--out', default='build/protected-debug1')
    if extra_arguments:
        extra_arguments(parser)
    args = parser.parse_args()
    stamp = datetime.datetime.now(datetime.timezone.utc).isoformat()
    base = (ROOT / 'docs/evidence').resolve()
```

tests/common.py | part 5

Complete source listing (wrapped)

```
172     destination = Path(os.environ.get('MONITOR_EVIDENCE_ROOT', 'docs/evidence/ve
    rificati
    on'))
173     destination = (ROOT / destination).resolve()
174     if destination == base or not destination.is_relative_to(base):
175         parser.error('MONITOR_EVIDENCE_ROOT must be a subdirectory of docs/evide
    nce')
176     evidence = destination if name == 'verification' else destination / 'tests'
    / name
177     if evidence.exists():
178         shutil.rmtree(evidence)
179     evidence.mkdir(parents=True)
180     code = 0
181     with (evidence / 'transcript.txt').open('w') as transcript:
182         tee = Tee(sys.stdout, transcript)
183         with contextlib.redirect_stdout(tee), contextlib.redirect_stderr(tee):
184             print(f'test={name} utc={stamp} cwd={ROOT}')
185             print('$', shlex.join(['python3', *sys.argv]))
186             print('uname=', os.uname())
187             try:
188                 function(args, evidence)
189                 status = 'PASS'
190             except PermissionError as error:
191                 code, status = 77, 'BLOCKED'
192                 print(f'BLOCKED: {error}; execution environment denied operation
    ')
193             except Exception:
194                 code, status = 1, 'FAIL'
195                 traceback.print_exc()
196             print(f'{status}: {name}')
197     (evidence / 'result.json').write_text(json.dumps({'test': name, 'status': st
    atus,
198                                                       'exit_code': code, 'started_
    utc': st
    amp,
199                                                       'finished_utc': datetime.dat
    etime.no
    w(datetime.timezone.utc).isoformat()}, indent=2) + '\n')
200     print('evidence:', evidence.relative_to(ROOT))
201     raise SystemExit(code)
```

tests/disconnect_probe.c | part 1

Complete source listing (wrapped)

```
 1  #include "protocol.h"
 2  #include <errno.h>
 3  #include <signal.h>
 4  #include <stdio.h>
 5  #include <sys/socket.h>
 6  #include <unistd.h>
 7
 8  int main(void)
 9  {
10      /* 使用真正的已關閉對端，確保 SIGPIPE 對照不依賴 TCP 時序。 */
11      struct sigaction action = {.sa_handler = SIG_DFL};
12      sigemptyset(&action.sa_mask);
13      if (sigaction(SIGPIPE, &action, NULL) < 0) return 2;
14      int pair[2];
15      if (socketpair(AF_UNIX, SOCK_STREAM, 0, pair) < 0) { perror("socketpair"); r
   eturn 2;
    }
16      close(pair[1]);
17      int result = mon_send_all(pair[0], "probe\n", 6, 1000);
18      int saved = errno;
19      close(pair[0]);
20      printf("result=%d errno=%d EPIPE=%d\n", result, saved, EPIPE);
21      return result == MON_IO_ERROR && saved == EPIPE ? 0 : 1;
22  }
```

tests/fault_proc.c | part 1

Complete source listing (wrapped)

```
 1  #include <errno.h>
 2  #include <stdio.h>
 3  #include <stdlib.h>
 4  #include <string.h>
 5
 6  FILE *__real_fopen(const char *path, const char *mode);
 7  FILE *__wrap_fopen(const char *path, const char *mode);
 8
 9  FILE *__wrap_fopen(const char *path, const char *mode)
10  {
11      /* 僅測試用連結器替身，不更動真實 /proc 或正式程式。 */
12      const char *failed_path = getenv("MONITOR_FAIL_PROC");
13      if (failed_path && strcmp(failed_path, path) == 0) {
14          errno = EACCES;
15          return NULL;
16      }
17      return __real_fopen(path, mode);
18  }
```

tests/io_probe.c | part 1

Complete source listing (wrapped)

```
#include "protocol.h"
#include <errno.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <sys/socket.h>
#include <sys/wait.h>
#include <unistd.h>

int main(void)
{
    const size_t length = 1024 * 1024;
    char *payload = malloc(length);
    if (!payload) return 2;
    memset(payload, 'Q', length);
    int pair[2], small_buffer = 4096;
    if (socketpair(AF_UNIX, SOCK_STREAM, 0, pair) < 0) { perror("socketpair"); f
ree(payl
oad); return 2; }
    if (setsockopt(pair[0], SOL_SOCKET, SO_SNDBUF, &small_buffer, sizeof(small_b
uffer))
< 0) return 2;
    pid_t child = fork();
    if (child < 0) return 2;
    if (child == 0) {
        close(pair[0]);
        size_t received = 0;
        char block[997];
        for (;;) {
            ssize_t n = read(pair[1], block, sizeof(block));
            if (n < 0 && errno == EINTR) continue;
            if (n < 0) _exit(3);
            if (n == 0) break;
            for (ssize_t i = 0; i < n; ++i) if (block[i] != 'Q') _exit(4);
            received += (size_t)n;
        }
        close(pair[1]);
        free(payload);
        _exit(received == length ? 0 : 5);
    }
    close(pair[1]);
    int result = mon_send_all(pair[0], payload, length, 3000);
    close(pair[0]);
    int status;
    while (waitpid(child, &status, 0) < 0) if (errno != EINTR) return 2;
    if (result != MON_LINE || !WIFEXITED(status) || WEXITSTATUS(status) != 0) re
turn 1;
```

tests/io_probe.c | part 2

Complete source listing (wrapped)

```
43      printf("partial writes: sent and verified %zu bytes; SO_SNDBUF requested=%d\
   n", leng
   th, small_buffer);
44      if (socketpair(AF_UNIX, SOCK_STREAM, 0, pair) < 0) return 2;
45      if (setsockopt(pair[0], SOL_SOCKET, SO_SNDBUF, &small_buffer, sizeof(small_b
   uffer))
   < 0) return 2;
46      int64_t start = mon_now_ms();
47      result = mon_send_all(pair[0], payload, length, 100);
48      long long elapsed = (long long)(mon_now_ms() - start);
49      close(pair[0]); close(pair[1]); free(payload);
50      printf("blocked writer: result=%d elapsed_ms=%lld\n", result, elapsed);
51      return result == MON_TIMEOUT && elapsed >= 90 && elapsed < 1500 ? 0 : 1;
52  }
```

tests/test_basic.py | part 1

Complete source listing (wrapped)

```
 1  import signal
 2  import socket
 3  import subprocess
 4  import time
 5  from common import ROOT, Server, command, fields, receive, request, run_test, wa
   it_for
 6
 7
 8  def test(args, evidence):
 9      command(['ldd', ROOT / args.out / 'bin/client'])
10      command(['ldd', ROOT / args.out / 'bin/server'])
11      with Server(args.out, evidence) as server:
12          for name in ['GET_STATUS', 'GET_CPU', 'GET_MEM', 'GET_UPTIME']:
13              result = server.client('--command', name, '--log-level', '2')
14              assert result.stdout.startswith('OK ')
15              values = fields(result.stdout)
16              if name == 'GET_CPU':
17                  assert 0 <= float(values['usage_pct']) <= 100
18                  assert int(values['sample_ms']) >= 190
19              if name == 'GET_MEM':
20                  proc = dict((line.split(':')[0], int(line.split()[1]))
21                              for line in open('/proc/meminfo') if line.startswith
   (('MemTo
   tal:', 'MemAvailable:')))
22                  total, available, used = (int(values[key]) for key in ['total_kb
   ', 'avai
   lable_kb', 'used_kb'])
23                  assert total == proc['MemTotal'] and 0 <= available <= total
24                  assert used == total - available
25                  assert abs(float(values['used_pct']) - 100 * used / total) < .02
26                  print(f'/proc comparison total={total}; available_snapshot={proc
   ["MemAva
   ilable"]}; response={available}')
27                  assert abs(available - proc['MemAvailable']) < max(total * .1, 1
   024)
28              if name == 'GET_UPTIME':
29                  proc_uptime = float(open('/proc/uptime').read().split()[0])
30                  assert abs(float(values['seconds']) - proc_uptime) < 2
31          with server.connect() as sock:
32              sock.sendall(b'GET_')
33              time.sleep(.05)
34              assert request(sock, b'MEM\n').startswith('OK MEM ')
35              sock.sendall(b'GET_UPTIME\nGET_MEM\n')
36              responses = [receive(sock), receive(sock)]
37              print('pipelined:', responses)
38              assert responses[0].startswith('OK UPTIME ') and responses[1].starts
   with('OK
    MEM ')
39              for invalid in [b'GET_MEM junk\n', b'\n', b'get_mem\n']:
40                  assert request(sock, invalid) == 'ERR INVALID_REQUEST\n'
```

tests/test_basic.py | part 2

Complete source listing (wrapped)

```
        for invalid in [b'GET_MEM\x00\n', b'GET_MEM\r\n', b'GET_MEM\x01\n', b'A'
 * 128 +
 b'\n']:
            with server.connect() as sock:
                assert request(sock, invalid).startswith('ERR ')
        # 邊界：127 字元加換行合法入框，128 字元加換行超限。
        with server.connect() as sock:
            assert request(sock, b'A' * 127 + b'\n') == 'ERR INVALID_REQUEST\n'
        with server.connect() as sock:
            sock.sendall(b'GET_MEM')
            sock.shutdown(socket.SHUT_WR)
            assert receive(sock) == 'ERR INVALID_REQUEST\n'
        server.client(input='1\n2\n3\n4\n6\n')
        result = server.client('--monitor', '--count', '3', '--interval-ms', '30
0')
        assert result.stdout.count('OK STATUS') == 3
        path = evidence / 'ctrl-c-client.stdout'
        with path.open('w') as output:
            argv = [str(ROOT / args.out / 'bin/client'), '--port', str(server.po
rt), '--
monitor', '--interval-ms', '300']
            print('$', ' '.join(argv), '> ', path.name)
            process = subprocess.Popen(argv, stdout=output, stderr=output)
            try:
                wait_for(lambda: 'OK STATUS' in path.read_text())
                process.send_signal(signal.SIGINT)
                assert process.wait(timeout=2) == 0
            finally:
                if process.poll() is None:
                    process.kill(); process.wait()
        print(path.read_text())
        invalid = server.client('--command', 'BAD', check=False)
        assert invalid.returncode == 1 and 'ERR INVALID_REQUEST' in invalid.stdo
ut
    for program in ['client', 'server']:
        for option in [['--port', '0'], ['--log-level', '3'], ['--timeout-ms', '
abc']]:
            assert command([ROOT / args.out / 'bin' / program, *option], check=F
alse).re
turncode == 2


if __name__ == '__main__':
    run_test('basic', test)
```

tests/test_concurrency.py | part 1

Complete source listing (wrapped)

```
 1  from concurrent.futures import ThreadPoolExecutor
 2  import json
 3  from pathlib import Path
 4  import re
 5  import threading
 6  import time
 7  from common import Server, child_pids, receive, request, run_test, wait_for
 8
 9
10  def test(args, evidence):
11      with Server(args.out, evidence, timeout_ms=10000) as server:
12          sockets = []
13          try:
14              sockets = [server.connect() for _ in range(10)]
15              pids = wait_for(lambda: child_pids(server.process.pid) if len(child_
   pids(ser
   ver.process.pid)) == 10 else None)
16              print(f'10 simultaneous connections: parent={server.process.pid} chi
   ld_pids=
   {pids}')
17              for pid in pids:
18                  status = Path(f'/proc/{pid}/status').read_text()
19                  assert f'PPid:\t{server.process.pid}\n' in status
20                  assert not re.search(r'^State:\s+Z', status, re.M)
21              barrier = threading.Barrier(10)
22
23              def worker(index):
24                  sock = sockets[index]
25                  barrier.wait(timeout=4)
26                  start = time.monotonic()
27                  sock.sendall(b'GET_CPU\n')
28                  response = receive(sock)
29                  end = time.monotonic()
30                  assert response.startswith('OK CPU '), response
31                  return {'client': index, 'start': start, 'end': end, 'response':
    respons
   e.strip()}
32
33              with ThreadPoolExecutor(max_workers=10) as executor:
34                  rows = list(executor.map(worker, range(10)))
35              overlap = min(row['end'] for row in rows) - max(row['start'] for row
    in rows
   )
36              concurrent = max(row['end'] for row in rows) - min(row['start'] for
   row in r
   ows)
37              print(json.dumps(rows, indent=2))
38              assert overlap > .1, f'10 client intervals did not overlap sufficien
   tly: {ov
   erlap}'
```

tests/test_concurrency.py | part 2

Complete source listing (wrapped)

```
            wait_for(lambda: server.logs().count('request_end') >= 10)
            logs = server.logs()
            starts, ends = {}, {}
            for line in logs.splitlines():
                pid = re.search(r'pid=(\d+)', line)
                event = re.search(r'request_(start|end) mono_ms=(\d+)', line)
                if pid and event and int(pid[1]) in pids:
                    (starts if event[1] == 'start' else ends)[int(pid[1])] = int
(event[2
])
            assert len(starts) == len(ends) == 10, 'DEBUG=1 required for server
interval
 evidence'
            server_overlap_ms = min(ends.values()) - max(starts.values())
            assert server_overlap_ms > 100
        finally:
            for sock in sockets:
                sock.close()
        wait_for(lambda: not child_pids(server.process.pid))
        sequential_start = time.monotonic()
        with server.connect() as sock:
            for _ in range(10):
                assert request(sock, b'GET_CPU\n').startswith('OK CPU ')
        sequential = time.monotonic() - sequential_start
        print(f'concurrent_s={concurrent:.6f} sequential_s={sequential:.6f} '
              f'client_overlap_s={overlap:.6f} server_overlap_ms={server_overlap
_ms}')
        assert concurrent < sequential * .7
        (evidence / 'timings.json').write_text(json.dumps({'clients': rows, 'pid
s': pids
,
            'concurrent_s': concurrent, 'sequential_s': sequential,
            'server_starts_ms': starts, 'server_ends_ms': ends,
            'server_overlap_ms': server_overlap_ms}, indent=2) + '\n')
        wait_for(lambda: not child_pids(server.process.pid))
        print('all children reaped; parent still running:', server.process.poll(
) is Non
e)


if __name__ == '__main__':
    run_test('concurrency', test)
```

tests/test_io.py | part 1

Complete source listing (wrapped)

```
from common import ROOT, command, run_test


def test(args, evidence):
    command([ROOT / args.out / 'bin/io_probe'])


if __name__ == '__main__':
    run_test('io', test)
```

tests/test_lifecycle.py | part 1

Complete source listing (wrapped)

```
 1  from pathlib import Path
 2  import time
 3  from common import Server, child_pids, request, run_test, wait_for
 4
 5
 6  def test(args, evidence):
 7      with Server(args.out, evidence) as server:
 8          parent = server.process.pid
 9          before = len(list(Path(f'/proc/{parent}/fd').iterdir()))
10          for _ in range(30):
11              with server.connect() as sock:
12                  assert request(sock, b'GET_MEM\n').startswith('OK MEM ')
13          wait_for(lambda: not child_pids(parent))
14          after = len(list(Path(f'/proc/{parent}/fd').iterdir()))
15          print(f'parent={parent} fd_before={before} fd_after={after} children={ch
    ild_pids
    (parent)}')
16          assert after == before
17          held = [server.connect() for _ in range(10)]
18          try:
19              pids = wait_for(lambda: child_pids(parent) if len(child_pids(parent)
    ) == 10
    else None)
20              print('shutdown with held children:', pids)
21              # Child 只保留 stdin/out/err、accepted socket；不保留 listener／self
    -pipe。
22              for pid in pids:
23                  fds = {p.name: str(p.readlink()) for p in Path(f'/proc/{pid}/fd'
    ).iterdi
    r()}
24                  print(f'child={pid} descriptors={fds}')
25                  assert len(fds) == 4
26              server.process.terminate()
27              assert server.process.wait(timeout=3) == 0
28              assert all(not Path(f'/proc/{pid}').exists() for pid in pids)
29          finally:
30              for sock in held:
31                  sock.close()
32          port = server.port
33      with Server(args.out, evidence, port=port, label='server-restart') as restar
    ted:
34          restarted.client('--command', 'GET_STATUS')
35          print(f'restarted successfully on same port={port}')
36
37
38  if __name__ == '__main__':
39      run_test('lifecycle', test)
```

tests/test_logging.py | part 1

Complete source listing (wrapped)

```
 1  from common import Server, run_test, wait_for
 2
 3
 4  def test(args, evidence):
 5      for debug in [0, 1]:
 6          for level in [0, 1, 2]:
 7              print(f'=== compile DEBUG={debug}, server runtime={level} ===')
 8              with Server(f'build/protected-debug{debug}', evidence, level=level,
 9                          label=f'server-debug{debug}-level{level}') as server:
10                  # Client 與 Server 的層級獨立，逐一驗證每種組合。
11                  for client_level in [0, 1, 2]:
12                      result = server.client('--command', 'GET_MEM', '--log-level'
   , str(cl
   ient_level))
13                      assert (' INFO ' in result.stderr) == (client_level >= 1)
14                      assert (' DEBUG ' in result.stderr) == (debug == 1 and clien
   t_level
   == 2)
15                      error = server.client('--command', 'BAD', '--log-level', str
   (client_
   level), check=False)
16                      assert error.returncode == 1 and ' ERROR ' in error.stderr
17                  wait_for(lambda: 'invalid_command' in server.logs())
18              logs = server.logs()
19              print(logs)
20              assert ' ERROR ' in logs
21              assert (' INFO ' in logs) == (level >= 1)
22              assert (' DEBUG ' in logs) == (debug == 1 and level == 2)
23
24
25  if __name__ == '__main__':
26      run_test('logging', test)
```

tests/test_proc_errors.py | part 1

Complete source listing (wrapped)

```
 1  import os
 2  from common import Server, request, run_test
 3
 4
 5  def test(args, evidence):
 6      for path, failed, healthy in [('/proc/stat', 'GET_CPU', 'GET_MEM'),
 7                                    ('/proc/meminfo', 'GET_MEM', 'GET_UPTIME'),
 8                                    ('/proc/uptime', 'GET_UPTIME', 'GET_MEM')]:
 9          print(f'fault injection: fopen({path!r}) returns EACCES in test binary o
   nly')
10          env = dict(os.environ, MONITOR_FAIL_PROC=path)
11          label = {'/proc/stat': 'cpu', '/proc/meminfo': 'memory', '/proc/uptime':
    'uptime
   '}[path]
12          with Server(args.out, evidence, binary='server-fault', env=env,
13                      label=f'server-{label}-failure') as server:
14              with server.connect() as sock:
15                  assert request(sock, (failed + '\n').encode()) == 'ERR SYSTEM_IN
   FO\n'
16                  assert request(sock, b'GET_STATUS\n') == 'ERR SYSTEM_INFO\n'
17                  assert request(sock, (healthy + '\n').encode()).startswith('OK '
   )
18
19
20  if __name__ == '__main__':
21      run_test('proc-errors', test)
```

tests/test_robustness.py | part 1

Complete source listing (wrapped)

```
 1  import signal
 2  import socket
 3  import struct
 4  import time
 5  from common import ROOT, Server, child_pids, command, receive, request, run_test
   , wait_f
   or
 6
 7
 8  def test(args, evidence):
 9      for variant, out in [('baseline', args.baseline), ('protected', args.out)]:
10          print(f'=== {variant}: Test A invalid and oversized ===')
11          with Server(out, evidence, timeout_ms=400, label=f'server-{variant}') as
    server:
12              for payload in [b'GET_MEM garbage\n', b'GET_MEM' + b'A' * 200 + b'\n
   ']:
13                  with server.connect() as sock:
14                      result = request(sock, payload)
15                      assert result.startswith('OK MEM ' if variant == 'baseline'
   else 'ER
   R ')
16              print(f'=== {variant}: Test B silent client ===')
17              with server.connect() as sock:
18                  wait_for(lambda: len(child_pids(server.process.pid)) == 1)
19                  start = time.monotonic()
20                  time.sleep(.8)
21                  pids = child_pids(server.process.pid)
22                  print(f'observed_s={time.monotonic() - start:.3f} remaining_chil
   dren={pi
   ds}')
23                  if variant == 'baseline':
24                      assert len(pids) == 1
25                  else:
26                      assert not pids
27                      assert receive(sock) == 'ERR TIMEOUT\n'
28                      assert sock.recv(1) == b''
29              wait_for(lambda: not child_pids(server.process.pid))
30              if variant == 'protected':
31                  print('=== protected: slow partial request cannot reset deadline
    ===')
32                  with server.connect() as sock:
33                      sock.sendall(b'G'); time.sleep(.25); sock.sendall(b'E'); tim
   e.sleep(
   .3)
34                      assert receive(sock) == 'ERR TIMEOUT\n'
35                  wait_for(lambda: not child_pids(server.process.pid))
36              print(f'=== {variant}: Test C deterministic shared send path ===')
37              probe = command([ROOT / out / 'bin/disconnect_probe'], check=False)
38              assert probe.returncode == (-signal.SIGPIPE if variant == 'baseline'
    else 0)
39              print(f'=== {variant}: Test C TCP reset integration ===')
40              start_count = server.logs().count('request_start')
```

tests/test_robustness.py | part 2

Complete source listing (wrapped)

```
            with server.connect() as sock:
                sock.sendall(b'GET_CPU\n')
                wait_for(lambda: server.logs().count('request_start') > start_co
unt)
                sock.setsockopt(socket.SOL_SOCKET, socket.SO_LINGER, struct.pack
('ii', 1
, 0))
            wait_for(lambda: not child_pids(server.process.pid))
            print('TCP reset server log:\n' + server.logs())
            assert server.process.poll() is None
            server.client('--command', 'GET_MEM')


 if __name__ == '__main__':
     run_test('robustness', test, lambda parser: parser.add_argument('--baseline'
, defaul
t='build/baseline-debug1'))
```

tests/verify.py | part 1

Complete source listing (wrapped)

```
"""保留 clean build 與完整無特權測試的總紀錄。"""
import hashlib
import json
from common import ROOT, command, run_test


def test(args, evidence):
    sources = [ROOT / 'Makefile']
    for directory in ['client', 'server', 'shared', 'tests', 'scripts']:
        sources.extend(path for path in (ROOT / directory).rglob('*')
                       if path.suffix in {'.c', '.h', '.py', '.sh'})
    hashes = {str(path.relative_to(ROOT)): hashlib.sha256(path.read_bytes()).hexdigest()
              for path in sorted(sources)}
    (evidence / 'source-sha256.json').write_text(json.dumps(hashes, indent=2) + '\n')
    command(['gcc', '--version'])
    command(['make', '--version'])
    command(['make', 'clean'])
    build = command(['make', '-j2'])
    assert 'warning:' not in build.stderr
    results = command(['make', 'test-all'], timeout=120)
    assert 'warning:' not in results.stderr
    command(['gdb', '--nx', '--batch', '-ex', 'set debuginfod enabled off', '-ex', 'brea
k main', '-ex', 'run', '-ex', 'bt',
             '-ex', 'continue', '--args', ROOT / 'build/protected-debug1/bin/client', '-
-help'])


if __name__ == '__main__':
    run_test('verification', test)
```

Appendix: Evidence Index

These repository paths locate primary execution records, not a bibliography. The companion archive preserves their byte-exact contents.

Evidence	Location
Complete verification	docs/evidence/verification/transcript.txt
Concurrency intervals	docs/evidence/verification/tests/concurrency/timings.json
Robustness comparisons	docs/evidence/verification/tests/robustness/transcript.txt
Logging controls	docs/evidence/verification/tests/logging/transcript.txt
Lifecycle / FD observations	docs/evidence/verification/tests/lifecycle/transcript.txt
Capture and statistics	docs/evidence/packet-capture/monitor.pcap; tcpdump-stats.txt
Capture analysis	docs/evidence/packet-capture/analysis.json
Demonstration logs	docs/evidence/demo/*.log